Kubernetes Knowledge Base

RAG-oriented expert reference based on the official Kubernetes documentation

Document ID: kubernetes-knowledge-official-001
Scope: Kubernetes concepts, architecture, operations, networking, storage, configuration and security
Source: https://kubernetes.io/docs/home/
Currency: official pages consulted on 2026-10-03; Kubernetes docs cover the current and previous four versions

How to use this document

This is a compact, paraphrased knowledge source for retrieval-augmented generation. It is designed to answer conceptual and operational questions, identify relevant Kubernetes objects, and route the reader to the canonical official page. It is not a replacement for version-specific API reference, a cluster configuration, or a production runbook.

1. Kubernetes mental model

Kubernetes is an open source platform for managing containerized workloads and services. Its API is declarative: a user describes desired state, and controllers continuously work to make actual state converge toward it. Kubernetes provides service discovery, load balancing, storage orchestration, rollouts and rollbacks, self-healing, scaling, batch execution and extensibility.

Kubernetes is not an all-inclusive PaaS. It does not build source code, prescribe one CI/CD system, provide every application middleware, or mandate one logging, monitoring or alerting solution. Those capabilities are integrated through APIs and the wider ecosystem.

Core relation: user or automation → Kubernetes API → controllers → Pods on Nodes → Services and external integrations

2. Cluster architecture

Layer	Components	Primary responsibility
Control plane	kube-apiserver	Front end for the Kubernetes API; accepts and serves cluster objects.
Control plane	etcd	Consistent, highly available key-value store for cluster data.
Control plane	kube-scheduler	Selects a suitable Node for Pods that do not yet have one.
Control plane	kube-controller-manager	Runs control loops such as Node, Job and EndpointSlice controllers.
Optional control plane	cloud-controller-manager	Connects cloud-specific control logic when a cloud provider is used.
Node	kubelet	Ensures containers described by PodSpecs are running and healthy.
Node	kube-proxy	Maintains network rules for Service traffic; optional when equivalent proxying is provided.
Node	Container runtime	Manages container lifecycle through the CRI, for example containerd or CRI-O.
Add-ons	DNS, CNI, monitoring, logging	Cluster-level capabilities implemented with Kubernetes resources or external components.

A cluster consists of a control plane and worker Nodes. Nodes host Pods; the control plane makes global decisions and responds to cluster events. Deployment options include traditional processes, static Pods, self-hosted control planes and managed Kubernetes services.

3. API objects and declarative management

Objects are persistent records in the Kubernetes API. A manifest commonly contains apiVersion, kind, metadata, spec and status. spec describes desired state; status is reported by Kubernetes components. Labels identify and group objects; selectors match labels; annotations store non-identifying metadata.

Object	What it represents	Important relationship
Namespace	Logical scope for namespaced objects	Separates teams, environments or applications.
Pod	Smallest deployable compute object	One or more containers sharing network and storage context.
Deployment	Stateless application management	Manages ReplicaSets, which create and replace Pods.
StatefulSet	Workloads needing stable identity or storage	Associates ordered Pods with persistent identity.
DaemonSet	One Pod on each matching Node	Common for node agents, networking and telemetry.
Job / CronJob	Run-to-completion work	Job runs once; CronJob creates Jobs on a schedule.

Object	What it represents	Important relationship
Service	Stable network endpoint	Selects backend Pods and is backed by EndpointSlices.
ConfigMap / Secret	Non-confidential / confidential configuration	Consumed by Pods as environment variables, arguments or files.

Common ownership chain: Deployment → ReplicaSet → Pod. A Service does not own Pods; it selects them by labels. EndpointSlice objects describe the current Service backends.

4. Workloads and lifecycle

A Pod is an execution unit containing one or more containers. Containers in one Pod share a network namespace and can communicate through localhost; the Pod has a lifecycle and is not normally repaired in place after a node-level failure. Workload controllers create replacement Pods to restore the desired state.

Workload	Choose it when	Operational behavior
Deployment	Pods are interchangeable and the application is stateless	Rolling updates, rollback and replica management.
StatefulSet	Pods need stable identity or persistent storage	Stable ordinal identity and ordered lifecycle behavior.
DaemonSet	A node-local agent must run on matching Nodes	Schedules a Pod for each matching Node, including new Nodes.
Job	A task should finish	Creates Pods until the completion target is met.
CronJob	A Job must run periodically	Creates Jobs according to a cron schedule.

5. Services and networking

Each Pod receives a unique cluster-wide IP under the Kubernetes network model. Pods can communicate across Nodes without NAT, subject to intentional segmentation. A Service provides a stable IP or hostname for a changing set of backend Pods. Kubernetes automatically manages EndpointSlices for those backends.

Concept	Meaning	Use
ClusterIP	Internal stable Service address	Default Service type for in-cluster clients.
NodePort	Exposes a Service on a port on each Node	Basic external access when a node address is reachable.
LoadBalancer	Requests an external load balancer when supported	Cloud or provider-integrated external access.
Ingress	HTTP/HTTPS routing into Services	Protocol-aware routing by host and path; needs an Ingress controller.
Gateway API	Extensible traffic-routing API	Advanced and dynamic infrastructure and route management.
NetworkPolicy	Traffic rules for Pods and external destinations	Requires a network implementation that enforces the policy.
DNS	Service and Pod name resolution	Cluster DNS gives applications stable names for Services.

The Pod network and NetworkPolicy enforcement are normally supplied by a CNI/network implementation. The Service API is Kubernetes-defined, while kube-proxy or an integrated network implementation programs traffic routing.

6. Configuration and storage

ConfigMaps hold non-confidential key-value configuration. Secrets are intended for confidential values such as passwords, tokens and keys, but they still require appropriate access control and encryption practices. Both can be injected into Pods as environment variables, command-line arguments or mounted files.

Volumes provide storage to Pods. Ephemeral volumes follow the Pod lifecycle; PersistentVolumes represent storage that can outlive a Pod; PersistentVolumeClaims request storage; StorageClasses describe classes of storage and can enable dynamic provisioning. CSI drivers integrate external storage systems.

Configuration relation: ConfigMap or Secret → Pod volume or environment → container process
Storage relation: Pod → PersistentVolumeClaim → PersistentVolume → storage provider

7. Security fundamentals

Control	Purpose	Key question for an agent
API access control	Protect the Kubernetes API	Who can authenticate and which actions can they perform?
TLS and encryption	Protect data in transit and optionally at rest	Are control-plane and client connections protected?
RBAC	Authorize users, groups and ServiceAccounts	Which Role or ClusterRole is bound to this identity?
Secrets	Handle confidential configuration	Is access limited and is storage encryption configured?

Control	Purpose	Key question for an agent
Pod Security Standards	Constrain risky Pod behavior	Does the namespace enforce a suitable security profile?
NetworkPolicy	Restrict network flows	Which sources and destinations are allowed?
Admission control	Validate or mutate API requests	Which policy changes or rejects this object?
Audit logging	Record security-relevant API activity	Who changed what, when and through which API path?

Security is layered: API authentication and authorization, admission controls, workload isolation, network controls, secure image/runtime practices and auditability all contribute. Never treat a Secret object alone as proof that a value is safe.

8. Resource management, scheduling and health

The scheduler considers resource requirements, affinity and anti-affinity, data locality, hardware/software constraints, policy and deadlines when choosing a Node. Containers should declare CPU and memory requests and limits appropriate to their workload. Probes communicate application health: readiness controls whether traffic is sent, liveness can trigger a restart, and startup gives slow applications time to initialize.

Horizontal Pod Autoscaling changes replica count based on observed metrics. Taints and tolerations influence placement. PodDisruptionBudgets limit voluntary disruption. ResourceQuota and LimitRange can constrain namespace usage and defaults.

9. kubectl operational reference

```
Syntax: kubectl [command] [TYPE] [NAME] [flags]
Inspect: kubectl get pods -A | kubectl describe pod NAME | kubectl get events -A
Apply: kubectl apply -f manifest.yaml | kubectl diff -f manifest.yaml
Debug: kubectl logs POD | kubectl exec -it POD -- COMMAND | kubectl port-forward svc/SVC LOCAL:REMOTE
Rollouts: kubectl rollout status deployment/NAME | kubectl rollout undo deployment/NAME
Context: kubectl config current-context | kubectl config set-context --current --namespace=NAME
Discovery: kubectl api-resources | kubectl explain deployment.spec.template.spec
```

Use namespace flags explicitly for automation. Prefer apply and diff for declarative changes. Treat delete, edit, exec, patch, scale, drain and rollout undo as potentially impactful operations that require authorization and context checks.

10. Observability and troubleshooting path

Kubernetes documentation treats metrics, logs and traces as distinct observability signals. A practical investigation starts with object state and events, then checks controller status, Pod conditions, container logs, resource usage, network reachability and recent configuration or rollout changes. Monitoring and logging integrations are pluggable; Kubernetes does not mandate one vendor or stack.

```
Recommended sequence: context/namespace → get object status → describe and events → logs → probes and resources → Service/EndpointSlice/DNS → recent rollout/configuration → policy and node health
```

11. RAG retrieval guide

Entity	Aliases	Relations to index
Pod	pods, po	runs containers; belongs to a namespace; scheduled to a Node
Deployment	deploy	owns ReplicaSets; manages stateless Pods
Service	svc	selects Pods; has ClusterIP/NodePort/LoadBalancer behavior; uses EndpointSlices
Node	nodes, no	runs Pods; has kubelet, runtime and possibly kube-proxy
ConfigMap	cm	stores non-confidential configuration consumed by Pods
Secret	secrets	stores confidential configuration; access must be controlled
PersistentVolumeClaim	pvc	requests storage and binds to a PersistentVolume
NetworkPolicy	netpol	controls allowed traffic when enforced by the network plugin

Chunking recommendation: preserve each numbered section as a parent document, then split tables and command blocks into child chunks with metadata fields source_url, section, entity, operation, version_scope and risk_level. For destructive or privileged questions, retrieve the security and kubectl safety sections together.

12. Official sources

The summaries above are paraphrases. Consult the canonical pages for exact syntax, version behavior and implementation details.

Documentation home: https://kubernetes.io/docs/home/

Overview: https://kubernetes.io/docs/concepts/overview/

Cluster architecture: https://kubernetes.io/docs/concepts/architecture/

Workloads: https://kubernetes.io/docs/concepts/workloads/

Services and networking: https://kubernetes.io/docs/concepts/services-networking/

Configuration: https://kubernetes.io/docs/concepts/configuration/

Storage: https://kubernetes.io/docs/concepts/storage/

Security: https://kubernetes.io/docs/concepts/security/

kubectl: https://kubernetes.io/docs/reference/kubectl/

Install tools: https://kubernetes.io/docs/tasks/tools/

Observability: https://kubernetes.io/docs/concepts/cluster-administration/observability/

Glossary: https://kubernetes.io/docs/reference/glossary/

Kubernetes API reference: https://kubernetes.io/docs/reference/kubernetes-api/

License: https://github.com/kubernetes/website/blob/main/LICENSE

Attribution: Kubernetes documentation content is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). This file is an original concise summary and link index, not a verbatim mirror.